# Plantasya Mobile Application

System Maintenance & IT Support Comprehensive Guide

## 1. System Architecture Overview

**Plantasya** is an offline-first Android application designed for plant identification, cataloging, custom care task scheduling, and 3D visualization.

| Component | Technology / Specification |
|---|---|
| Language & SDK | Kotlin / Android SDK (Min SDK 24, Target SDK 35) |
| Architecture Pattern | MVVM (Model-View-ViewModel) + Coroutines & Flow |
| Database Engine | Room Persistence Library (SQLite DB: plantasya_db, Version 12) |
| Machine Learning | TensorFlow Lite (ModelUnquant.tflite) + CameraX Engine |
| 3D Graphics Engine | Google Filament 3D Graphics Library (1.49.2) |
| Scheduling System | AlarmManager (Exact Alarms) + NotificationReceiver |

## 2. Preventive Maintenance (Proactive Health & Reliability)

### 2.1 Database Indexing & Vacuuming

- **Database Auto-population & Fragmentation:** Room operations store plant photos as binary blobs (BLOB). Inserting/deleting plants or tasks causes SQLite fragmentation over time.
- **Monthly Vacuuming Routine:** Execute SQLite VACUUM through a migration or admin debug utility to reclaim unused storage and optimize query performance.
- **Foreign Key Verification:** Cascade deletion is enabled on OwnedPlant → Task relations (ForeignKey.CASCADE). Ensure orphaned tasks do not accumulate.

### 2.2 Asset & Memory Lifecycle Hygiene

- **Filament 3D Memory Hygiene:** Filament in ThreeDPlantView.kt allocates native C++ memory buffers (ModelViewer, Engine, Skybox). Ensure Choreographer.removeFrameCallback and engine.destroyEntity are executed during Activity onDestroy().
- **Image Compression Limits:** Image conversions inside BitmapConverter.kt convert bitmaps to byte arrays. Keep cover photos under 300 KB to avoid Room SQLite cursor size limits (2 MB Window Cursor limit).

### 2.3 Alarm & Exact Notification Scheduling

- **Android 12+ (API 31+) Exact Alarm Audits:** Exact alarms require SCHEDULE_EXACT_ALARM permissions. Verify that TaskScheduler.kt gracefully falls back to setAndAllowWhileIdle() if permission is denied.

# 3. Corrective Maintenance (Troubleshooting & Remediation)

### 3.1 Machine Learning Scanner Failures (TFLite / CameraX)

| Symptom | Probable Cause | Corrective Action |
|---|---|---|
| Scan returns 'Unknown' | Low confidence threshold (<80%) or unnormalized bitmap. | 1. Check ScanActivity.kt → confidenceThreshold = 0.80f. Adjust to 0.70f if overly strict.<br>2. Ensure ImageProcessor applies ResizeOp(224, 224) and NormalizeOp(127.5f, 127.5f). |
| IndexOutOfBoundsExcepti on | Output tensor labels mismatched with labels array. | Ensure labels array in ScanActivity.kt exactly matches TFLite output classes (25 classes, indices 0 to 24). |

### 3.2 3D Model Rendering Issues (ThreeDPlantView.kt)

| Symptom | Probable Cause | Corrective Action |
|---|---|---|
| Toast: 'Model for [Plant] not found' | Mismatch between plant name in DB and .glb asset filename. | Ensure ThreeDPlantView.getModelFileName(plantName) contains the mapping rule for the plant name. |
| Black screen or invisible 3D Model | Studio lighting missing or uninitialized Filament engine. | 1. Verify Utils.init() is called in ThreeDPlantView.<br>2. Check setupStudioLighting() intensity values (40,000.0f - 120,000.0f). |

# 4. Emergency Maintenance (Incident Playbook)

**■ EMERGENCY WORKFLOW PROCEDURES**

**1. ADMIN CREDENTIAL RECOVERY:** If administrator credentials are lost, trigger app data clear or update populateDatabase() inside AppDatabase.kt.
**Default Fallback Credentials:** Username: admin | Password: adminplantasya (SHA-256 Digest: 0507a67137f814981883582496a84f3c78a9c79234b358c568f44d1808620863)

**2. ALARM REMINDER DEADLOCK RECOVERY:** If reminders stop triggering after a system crash or OS update, force broadcast the boot receiver using ADB:
adb shell am broadcast -a android.intent.action.BOOT_COMPLETED -n com.example.plantasya_mobileapp/.NotificationReceiver

**3. CRITICAL DB CORRUPTION RESET:** Execute adb shell pm clear com.example.plantasya_mobileapp to purge local state and trigger automatic populateLibraryIfEmpty() bootstrap.

# 5. Component-Specific Operational Procedures

### 5.1 Updating the ML Plant Classification Model

1. Train new model on Teachable Machine / TensorFlow with 224x224 MobileNet architecture.
2. Replace ModelUnquant.tflite inside app/src/main/ml/.
3. Update labels list in ScanActivity.kt if new classes are added.
4. Update populateLibraryIfEmpty in AppDatabase.kt to include catalog information for new plants.

### 5.2 Task Logic & Overdue Rule Parameters

| Task Functionality | File Location | Key Variable / Function |
|---|---|---|
| Care Task Generator | TaskSyncManager.kt | getTasksForLibraryPlant() |
| Alarm Schedule Calculation | TaskScheduler.kt | calculateNextTime() |
| 24-Hour Overdue Threshold | TaskAdapter.kt, DashboardFrag.kt, NotificationReceiver.kt | OVERDUE_THRESHOLD_MILLIS = 86400000L |
| Lock Checkbox Until Sched | TaskAdapter.kt | holder.checkBox.isEnabled = !isScheduledForFuture |

## 6. Diagnostic Logcat Commands

Use the following ADB commands during debugging or system maintenance:

```
# Filter all Plantasya runtime logs:
adb logcat -s "Plantasya" "ModelViewer" "Filament" "Room"

# Filter exact alarm triggers:
adb logcat | grep "ACTION_TASK_REMINDER"

# Inspect active SQLite Database tables via ADB Shell:
adb shell run-as com.example.plantasya_mobileapp sqlite3 databases/plantasya_db "SELECT id_lib,
plant_name FROM library_plant_tbl;"
```